Web Application Security Assessment

Vulnerability Findings and Testing Evidence

Target: https://target.example.test
Scope: Five selected findings from the original assessment

About this report

This report records five findings from my web application security testing. For each finding, I describe what I tested, what I observed, why it matters and what should be changed. The screenshots are taken from the original assessment and are placed next to the reproduction step they support.

Sensitive account values, session tokens, target host details, local paths and identifiable certificate information have been masked. Screenshots that contain extensive personal or session data are fully blurred.

Selected findings

AUTHZ-001 - Broken Function Level Authorization (BFLA) (Critical)

INFO-03-001 - Production JavaScript Source Maps Publicly Accessible (Medium)

CONF-003 - Unauthenticated Spring Boot Actuator Endpoints (Medium)

AUTHZ-002 - Protected Email Address and Mobile Number Can Be Changed Through the Profile API (Medium)

AUTHZ-002 - IDOR / Broken Object Level Authorization in Certificate Access (High)

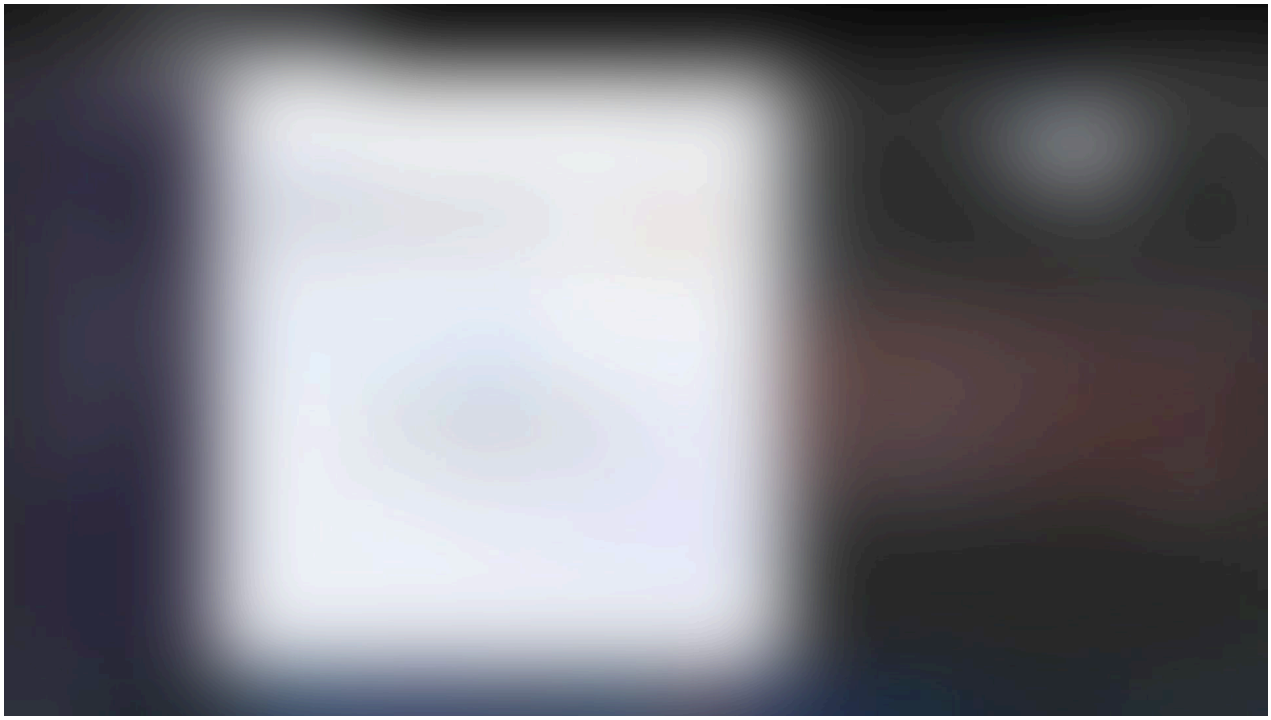
1. AUTHZ-001 - Broken Function Level Authorization (BFLA)

Severity: Critical

I found that the application used a role value stored in a browser-controlled cookie to decide which dashboard and navigation items to show. After changing the cookie from PARTICIPANT to ADMIN, the administrative interface appeared. I then tested an administrative Target Audience Management action. The server accepted the operation while the request was still authenticated with a participant JWT, which showed that this was not only a front-end display issue.

Steps to reproduce and evidence

Step 1 - Log in with a valid participant account and open the participant dashboard.

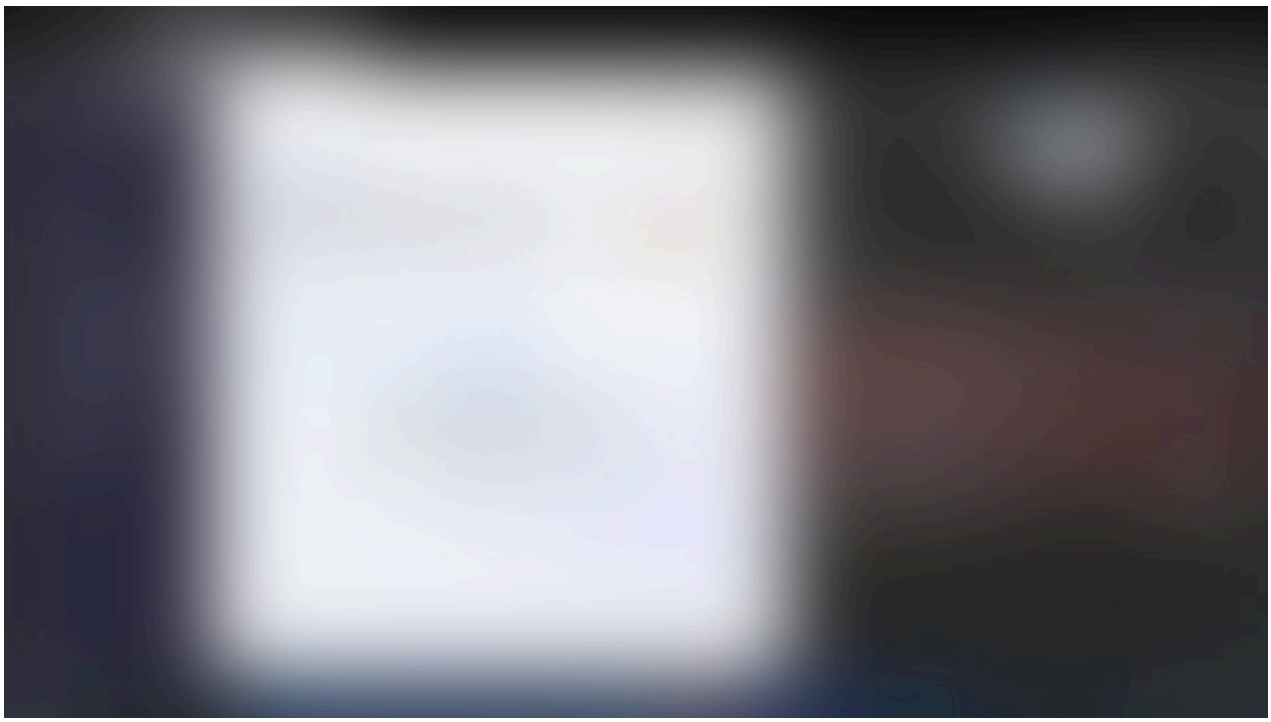


BFLA - Login baseline

Activity: Logged in with a valid participant account and opened the participant dashboard before testing role-based access.

Step 2 - Open Developer Tools, go to Application > Cookies and locate the role cookie.

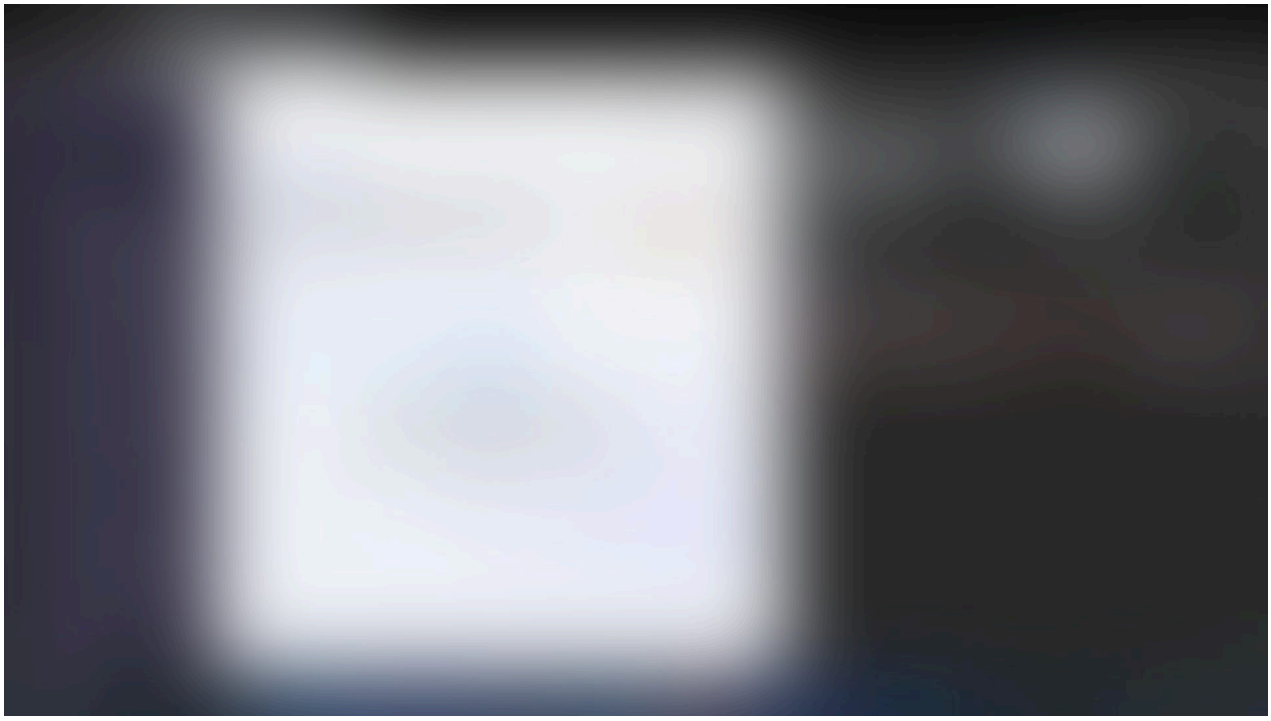
Step 3 - Change the role value from PARTICIPANT to ADMIN in the browser.



BFLA - Role cookie modified

Activity: Opened browser Developer Tools, located the role cookie and changed its value from PARTICIPANT to ADMIN. Cookie tokens and identifiers are masked.

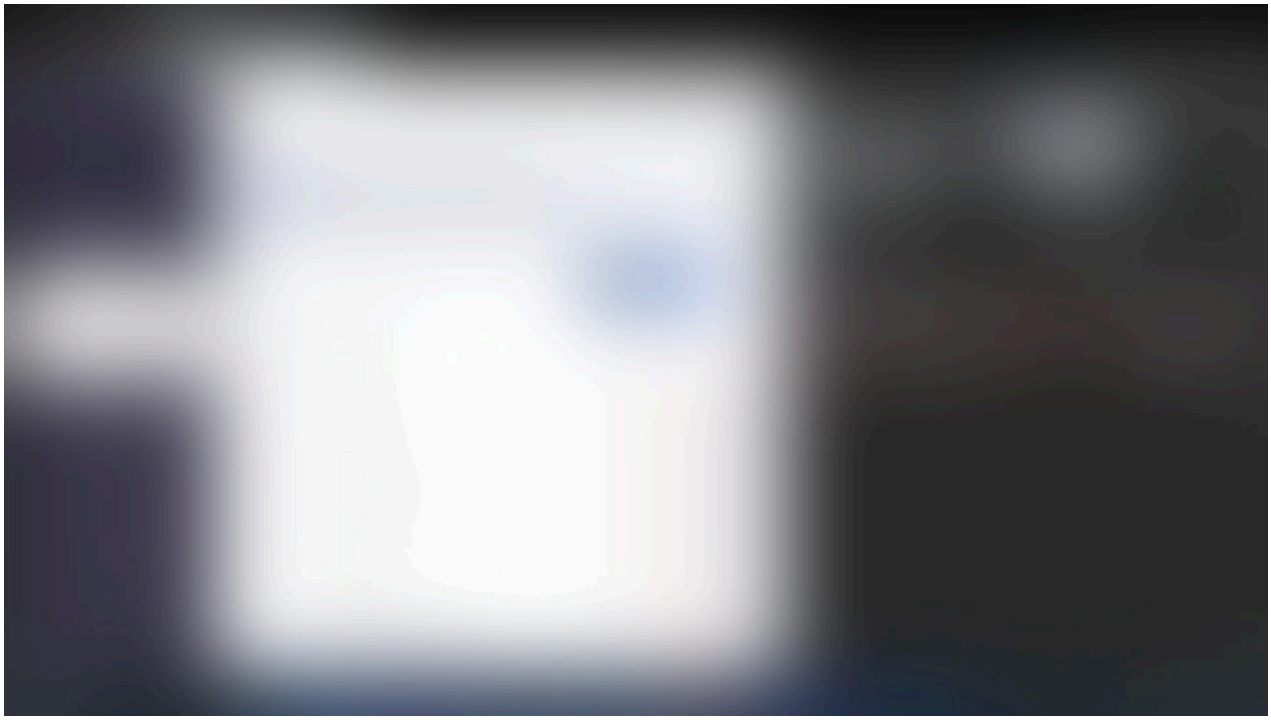
Step 4 - Refresh the page and check whether the administrative dashboard and menu items appear.



BFLA - Administrative dashboard displayed

Activity: Refreshed the page after changing the role value and observed that the administrative dashboard and menu items were rendered.

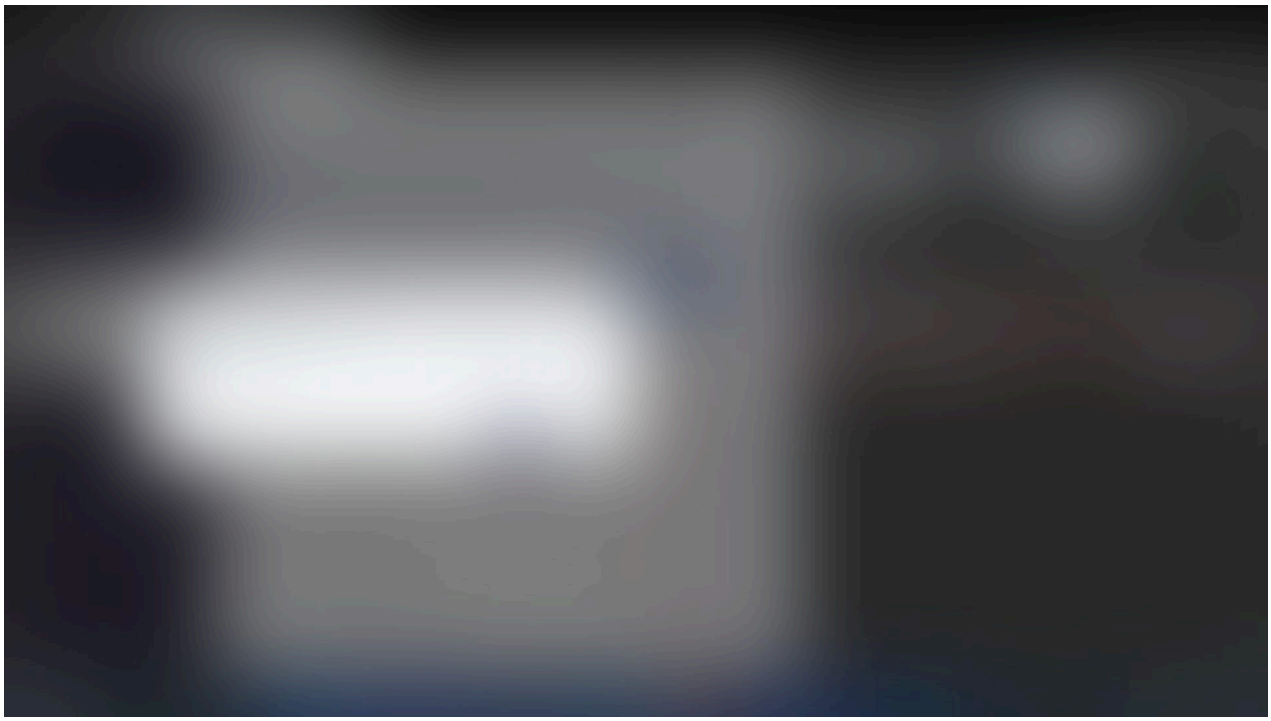
Step 5 - Open Manage Target Audience > Target Audience Category Management.



BFLA - Administrative page opened

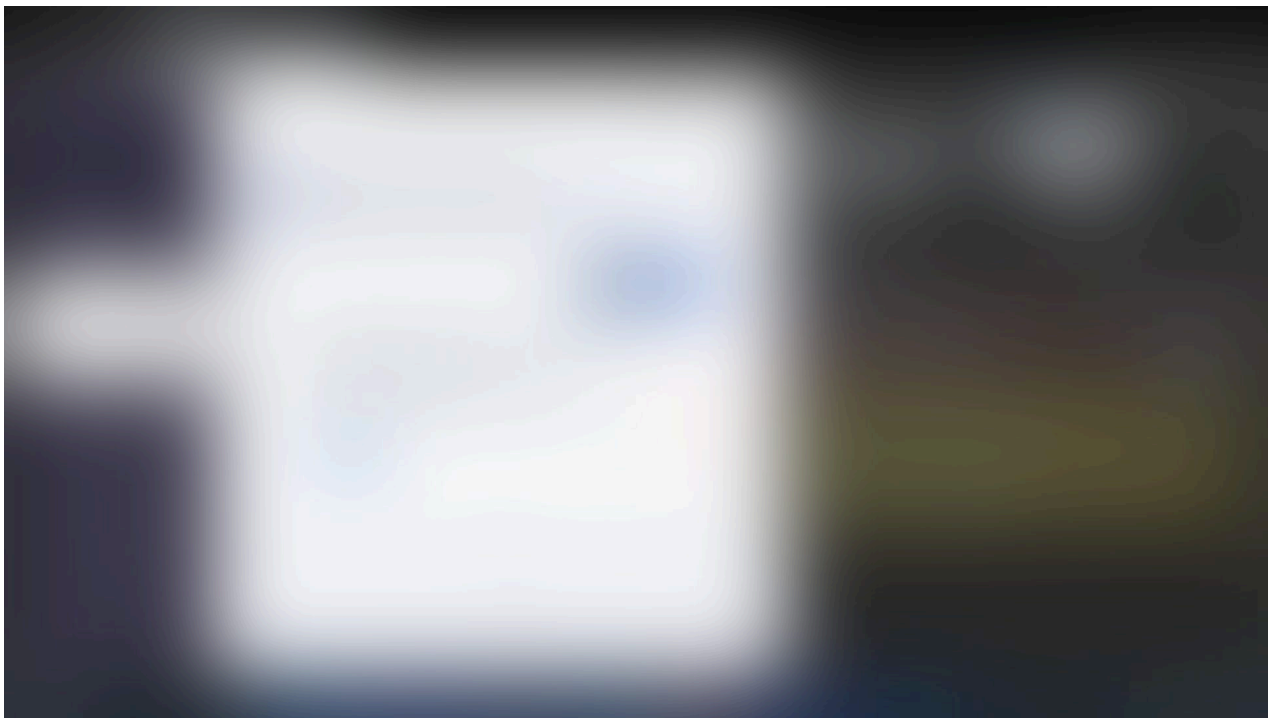
Activity: Opened Target Audience Category Management from the administrative navigation.

Step 6 - Open the category form and submit an administrative action. The original assessment recorded that the operation completed using the participant JWT.



BFLA - Category creation dialog

Activity: Opened the add-category dialog to test an administrative operation.



BFLA - Category operation completed

Activity: Submitted the test category action and observed the new category in the list. The original report states that this operation was accepted while using the participant JWT.

Impact

An authenticated participant could perform administrative operations and modify privileged application data. This creates a risk to the integrity of administrative records and configuration.

Root cause

The backend validated the JWT but did not consistently verify the user role before executing privileged operations. The frontend also trusted a role value that could be changed by the browser user.

Recommended remediation

Enforce server-side role checks on every administrative endpoint using the role from the validated authentication context. Do not rely on a client-controlled cookie for authorization decisions. Verify that participant requests to admin-only endpoints are rejected even if the interface is manipulated.

References

OWASP API Security Top 10 2023 - API5: Broken Function Level Authorization; CWE-285; WSTG-ATHZ-02 / WSTG-ATHZ-03

2. INFO-03-001 - Production JavaScript Source Maps Publicly Accessible

Severity: Medium

I found that source-map files associated with the production JavaScript bundle were publicly accessible. I requested the related map file and used it to reconstruct the original React source files. The recovered files included API modules, routes, dashboard components and authentication-related components.

Steps to reproduce and evidence

Step 1 - Browse the application and note the JavaScript bundle and related static assets.



Source maps - Public asset manifest inspected

Activity: Requested the public asset manifest and inspected the static asset references. The visible listing includes source-map file references; organization-specific asset labels are masked.

Step 2 - Request the corresponding .map file from the static asset path.

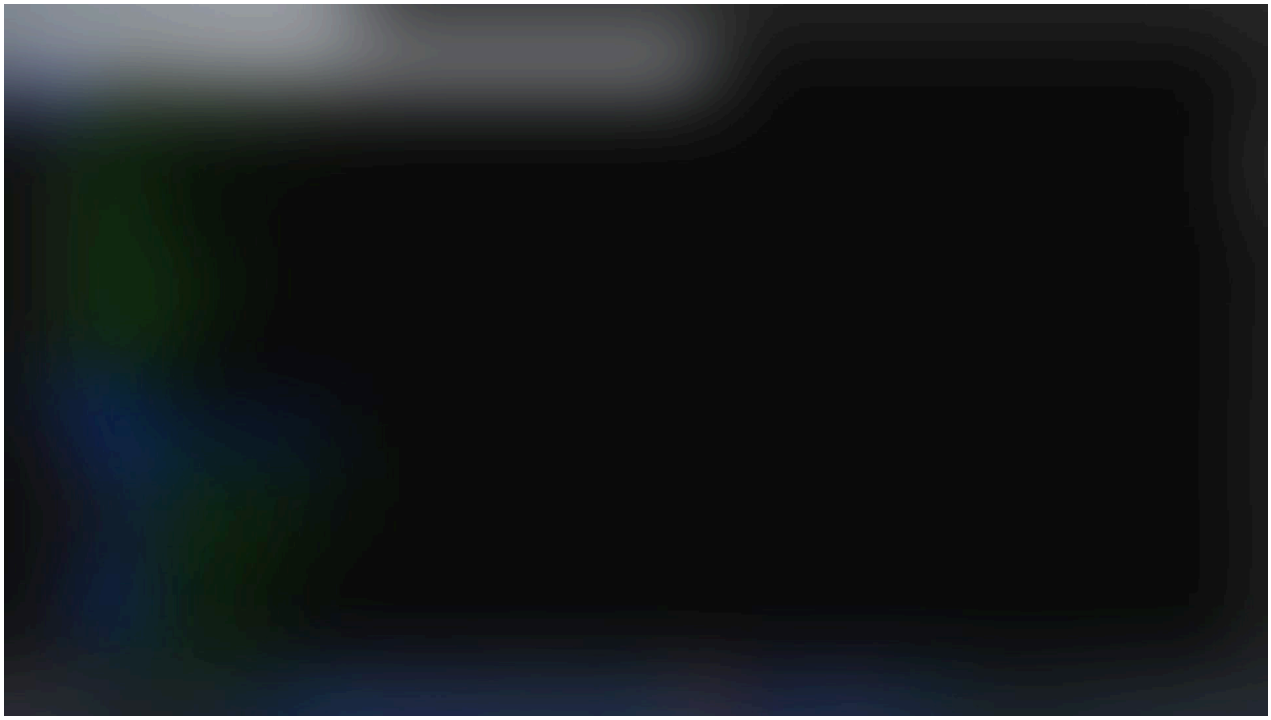
Step 3 - Observe that the server returns the source-map file successfully.



Source maps - Map request tested in Burp

Activity: Sent a GET request for the JavaScript source map in Burp Repeater and inspected the server response.

Step 4 - Use a source-map reconstruction tool to recover the original application source files.



Source maps - Recovered file tree

Activity: Ran the reconstruction command against the downloaded source map and reviewed the recovered source-file tree. The local username and path are masked.



Source maps - Authentication files located

Activity: Searched the recovered source entries for authentication and login-related components, then inspected the recovered AuthContext source.

Impact

An attacker could inspect client-side source files, API endpoint structures, authentication-related logic and application routing. In this assessment, the recovered source helped identify areas for further access-control testing.

Root cause

Source maps were generated for the production build and deployed alongside the public JavaScript bundle without access restrictions.

Recommended remediation

Do not publish source maps in production unless required. Remove existing public .map files and configure the production build or web server to prevent public access to them.

References

CWE-200; WSTG-CONF-05; React production build and source-map configuration documentation

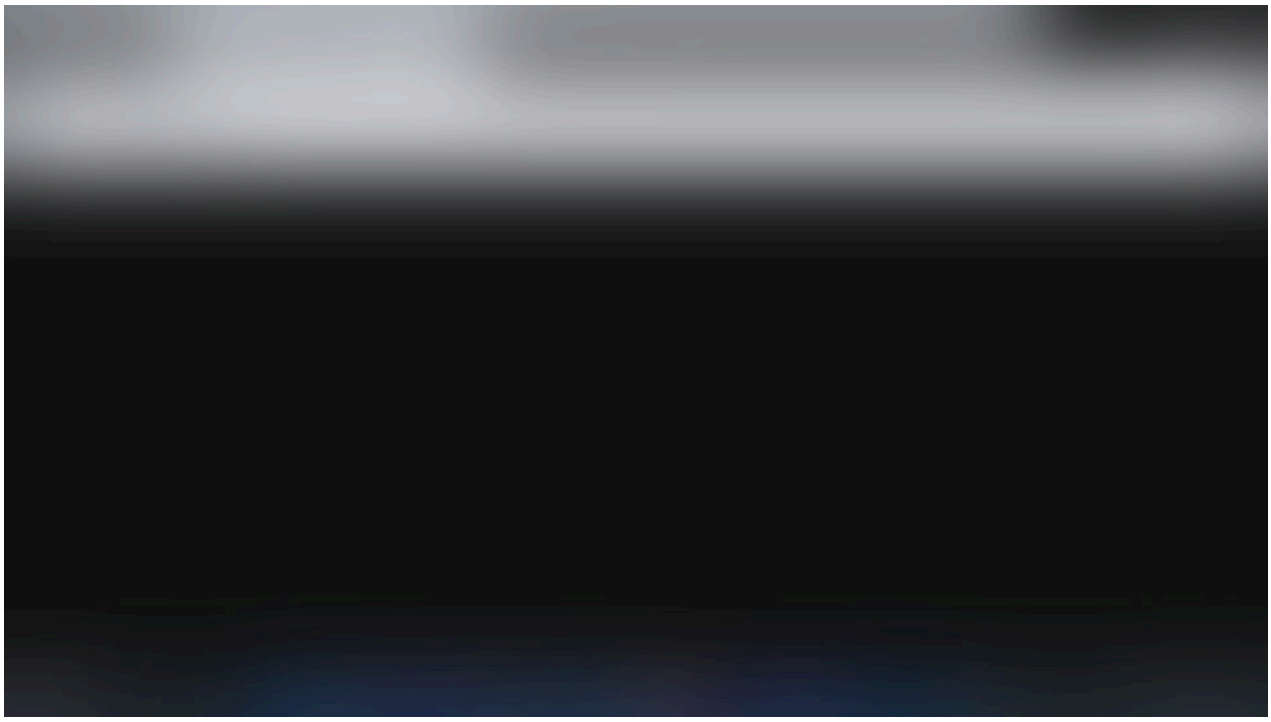
3. CONF-003 - Unauthenticated Spring Boot Actuator Endpoints

Severity: Medium

I tested Spring Boot Actuator management endpoints and found that the tested endpoints responded without authentication. The responses disclosed application health information, runtime metric names, internal service references and gateway route details.

Steps to reproduce and evidence

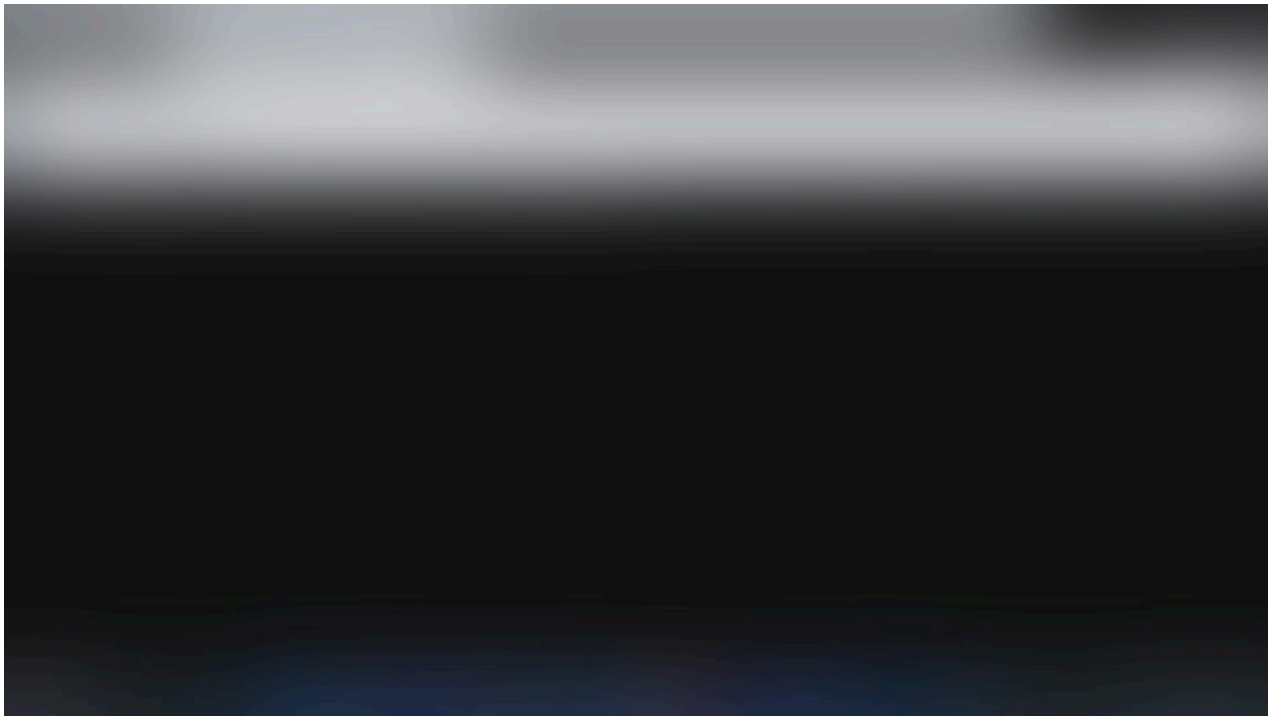
Step 1 - Send a GET request to /actuator/health without an authenticated session.



Actuator - Health endpoint response

Activity: Requested /actuator/health without logging in. The address bar in this screenshot visibly shows the health endpoint. The nearby original figure note calls it a metrics response, so this caption follows the visible URL.

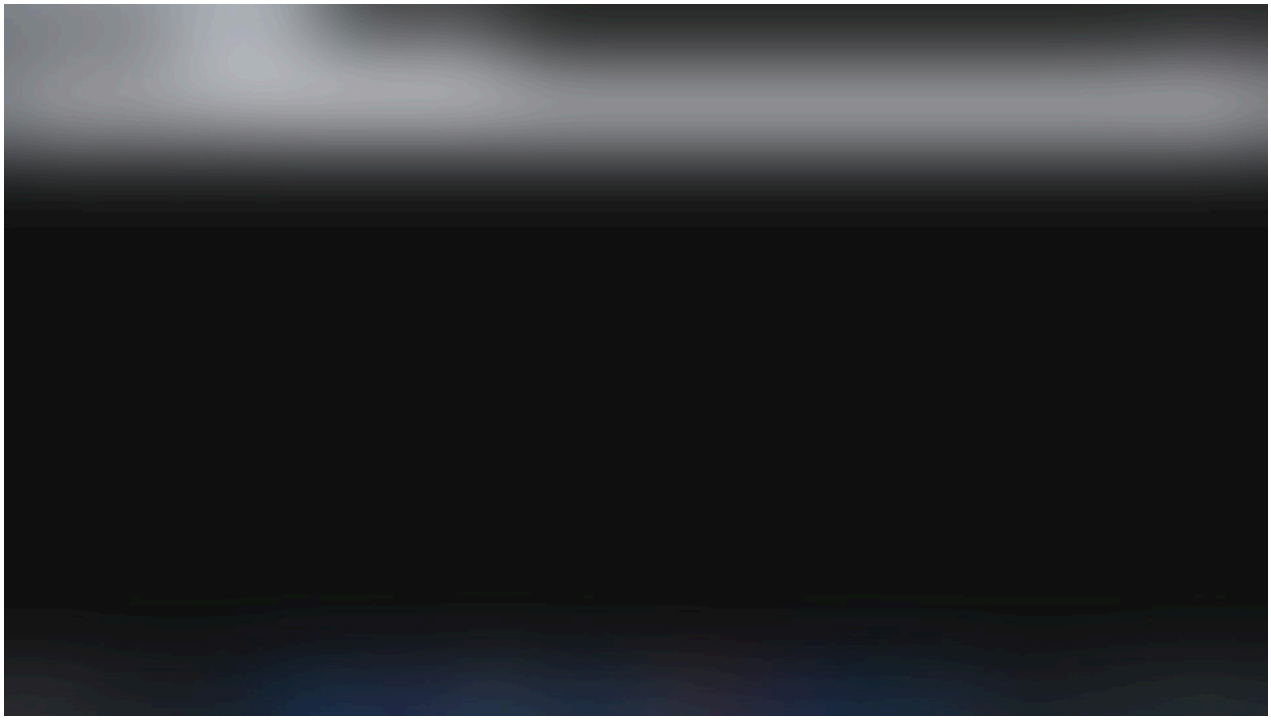
Step 2 - Send a GET request to /actuator/metrics and inspect the response.



Actuator - Metrics endpoint response

Activity: Requested /actuator/metrics without authentication. The response listed runtime metric names; the response details are masked to avoid exposing internal operational information.

Step 3 - Send a GET request to /actuator/ratelimiters and inspect the returned route information.



Actuator - Rate limiter endpoint response

Activity: Requested /actuator/ratelimiters without authentication. The response disclosed internal gateway routes and API paths, which are masked in this portfolio copy.

Step 4 - Confirm that the tested responses are returned without authentication or elevated privileges.

Impact

Exposed service names, API paths and runtime information can help an attacker understand the application architecture and identify further areas to test. This report does not claim that code execution was demonstrated through Actuator.

Root cause

Management endpoints were reachable without an authentication and authorization requirement, exposing operational information to unauthenticated users.

Recommended remediation

Require authentication and authorization for management endpoints. Disable unnecessary Actuator endpoints in production and expose only the minimum information needed by trusted monitoring systems.

References

CWE-200; Spring Boot Actuator security documentation; WSTG-CONF

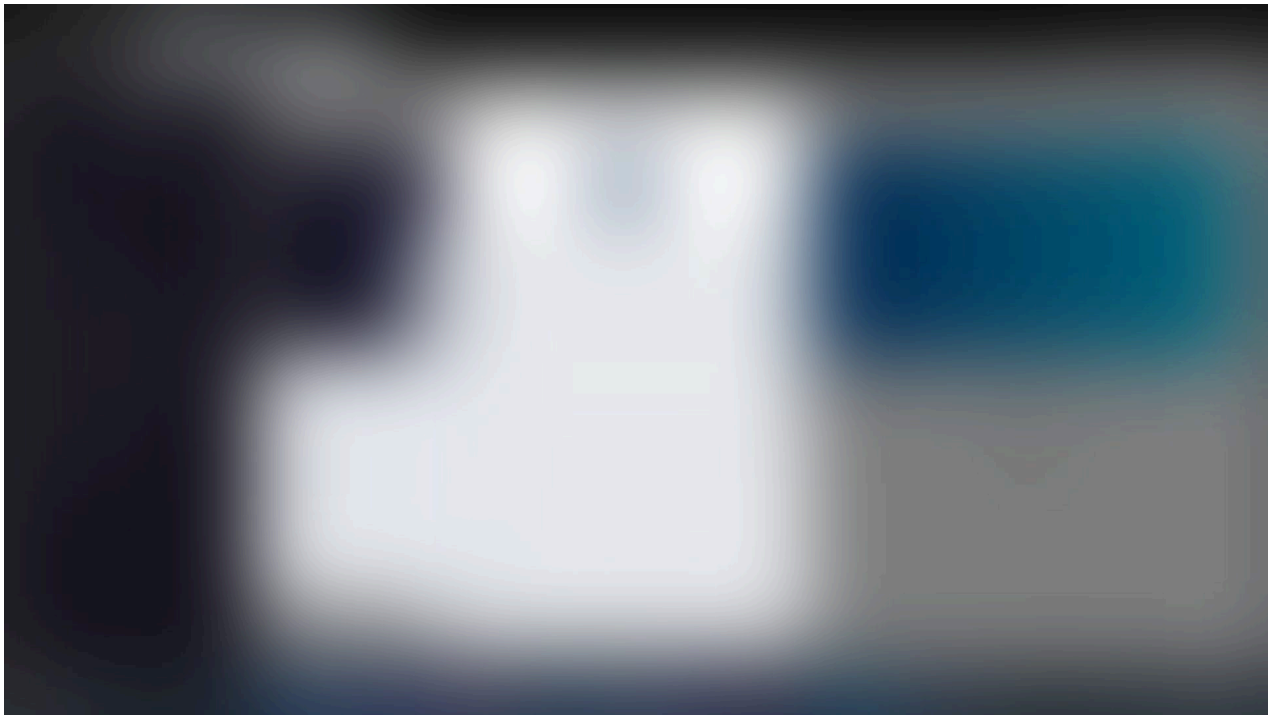
4. AUTHZ-002 - Protected Email Address and Mobile Number Can Be Changed Through the Profile API

Severity: Medium

The participant profile page presented the email address and mobile number as non-editable fields. I intercepted the profile update request and modified those values in the request. The server accepted the update. This showed that the restriction was present in the interface but was not enforced by the backend endpoint.

Steps to reproduce and evidence

Step 1 - Log in as a participant, open Profile and observe how the email and mobile fields are presented.



Profile API - Profile form inspected

Activity: Opened the participant profile editor and recorded how the email and mobile fields were presented in the interface. Personal values in the form and profile are masked.

Step 2 - Click Edit Profile and intercept the PUT /api/participant/update-profile request in Burp Suite.



Profile API - Update request intercepted

Activity: Intercepted the PUT /api/participant/update-profile request in Burp Suite. Request headers and the bearer token are obscured.

Step 3 - Modify the email and phone parameters, forward the request and review the server response.



Profile API - Modified request forwarded

Activity: Forwarded the modified profile request and reviewed the server response. This original screenshot contains account values and request data, so it is fully blurred.



Profile API - Mobile and email update observed

Activity: The original figure note identifies this evidence as showing the mobile-number update; the adjacent note also states that the email address was updated successfully. The screenshot is fully blurred because it contains personal data.

Step 4 - Restore the original email and mobile values after confirming the behavior.



Profile API - Original values restored

Activity: After confirming the update behavior, restored the original email and mobile values to retain access to the test account. The screenshot is fully blurred because it contains personal details.

Impact

An authenticated participant could change account identifiers that the interface was intended to protect. If these identifiers are used for login, verification or account recovery, changing them without a dedicated verification process can affect account integrity and recovery.

Root cause

The general profile update endpoint accepted email and mobile number changes. The application relied on disabled or non-editable interface fields instead of enforcing the restriction on the server.

Recommended remediation

Allow the general profile endpoint to update only approved profile fields. Reject changes to email and mobile through this endpoint and implement a separate verified workflow for changing authentication identifiers.

References

CWE-285; CWE-602; OWASP A01 Broken Access Control; WSTG-ATHZ-02 / WSTG-BUSL-01

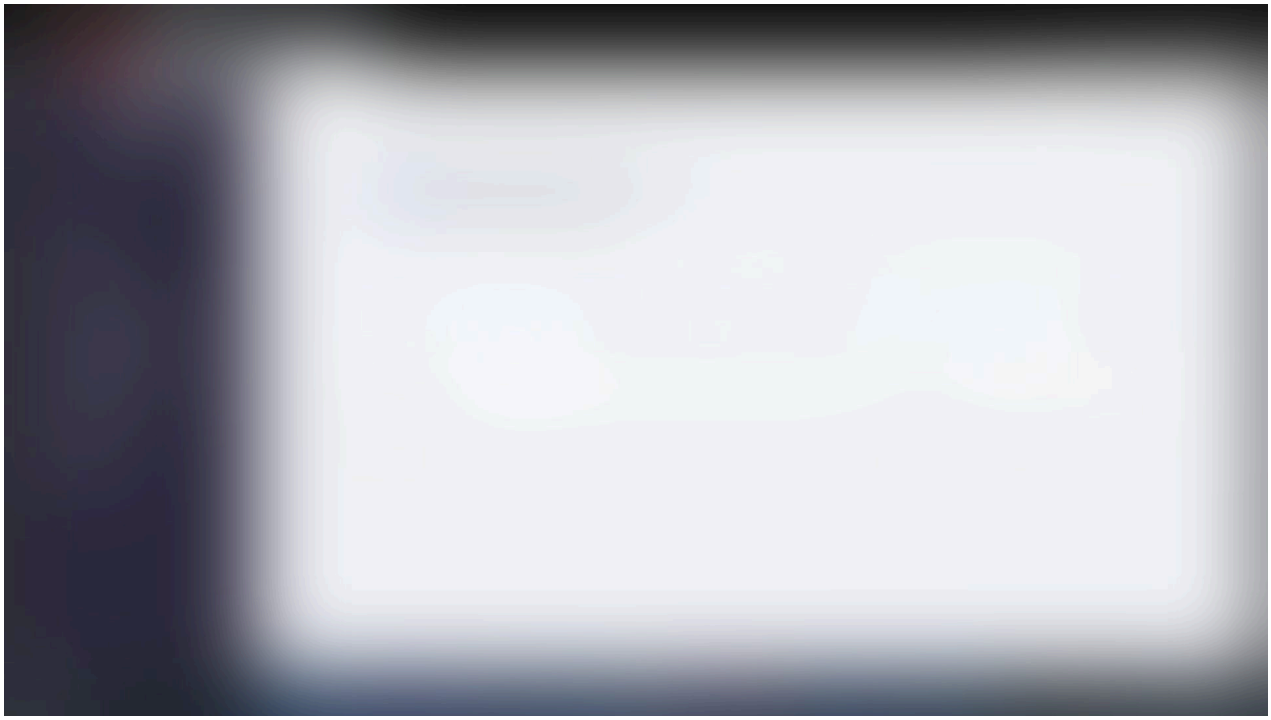
5. AUTHZ-002 - IDOR / Broken Object Level Authorization in Certificate Access

Severity: High

I tested the certificate download request and found that the certificate number was included in the request URL. I sent the request to Burp Repeater and changed the number while keeping the same authenticated session. The server returned certificates belonging to other participants. The endpoint checked whether a certificate number existed, but did not verify that the certificate belonged to the logged-in user.

Steps to reproduce and evidence

Step 1 - Log in with a normal participant account and open the enrolled-programmes dashboard.



Certificate IDOR - Participant dashboard

Activity: Logged in with a normal participant account and opened the enrolled-programmes dashboard where the certificate download control was available.

Step 2 - Click the certificate download button and intercept the outgoing request. The certificate number is part of the URL.



Certificate IDOR - Download request intercepted

Activity: Clicked the certificate download option and intercepted the GET request in Burp Suite. The target host, object identifier and session token are masked.

Step 3 - Send the request to Burp Repeater and inspect the server response.



Certificate IDOR - Request sent to Repeater

Activity: Sent the captured certificate request to Repeater and inspected the server response for the current certificate request. Request and response data are obscured because they contain session and certificate data.

Step 4 - Right-click the response and choose the option to open the response in a browser.



Certificate IDOR - Response menu opened

Activity: Right-clicked the response and selected the option to open it in a browser. Request and response content are obscured.

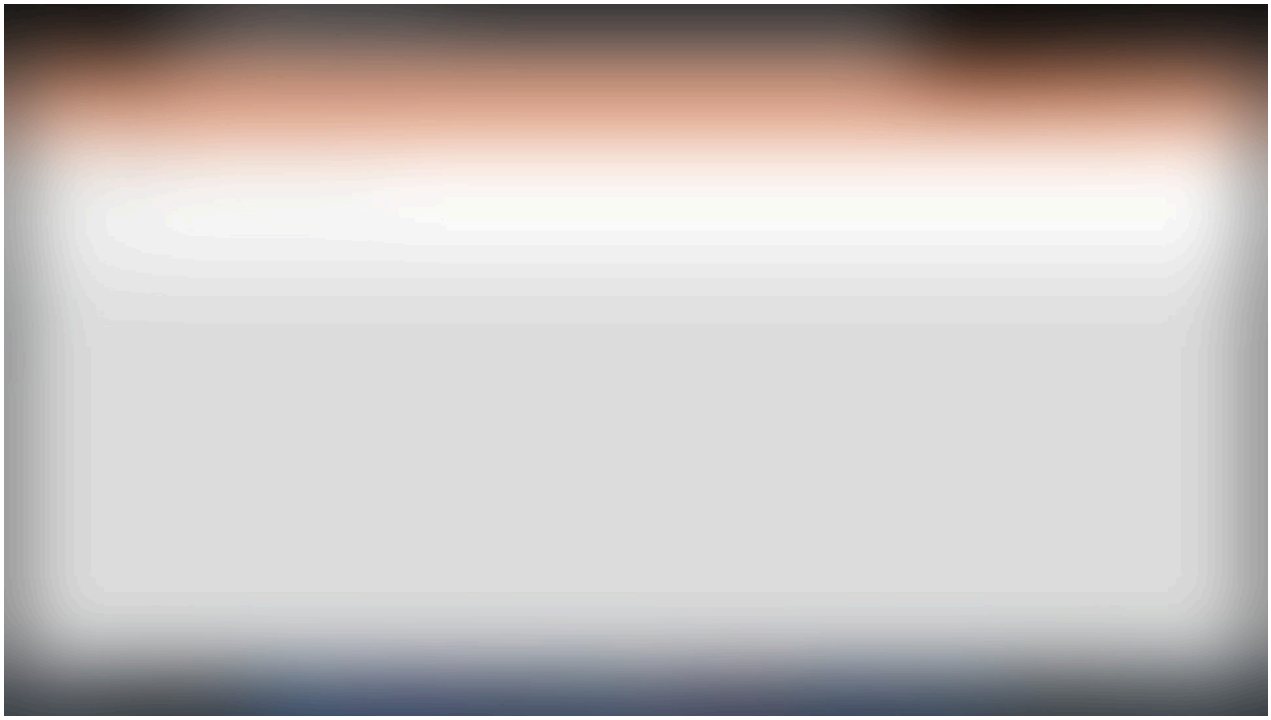
Step 5 - Use the temporary Burp-generated link to view the server response.



Certificate IDOR - Temporary response link

Activity: Viewed the temporary link generated by Burp for opening the server response in a browser. The entire screenshot is blurred because it contains a temporary access URL and response context.

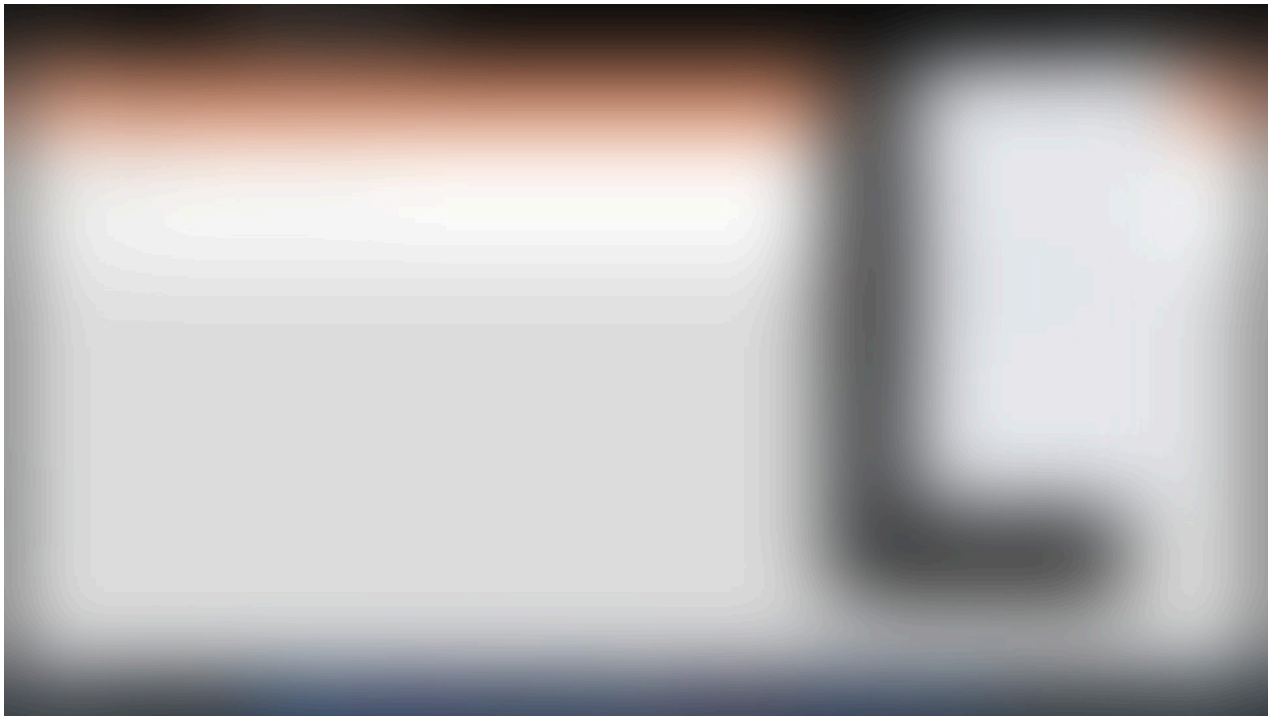
Step 6 - Open the link in a browser and select Show response.



Certificate IDOR - Browser response page

Activity: Opened the Burp response page in a browser and used the Show response control. The temporary URL is masked.

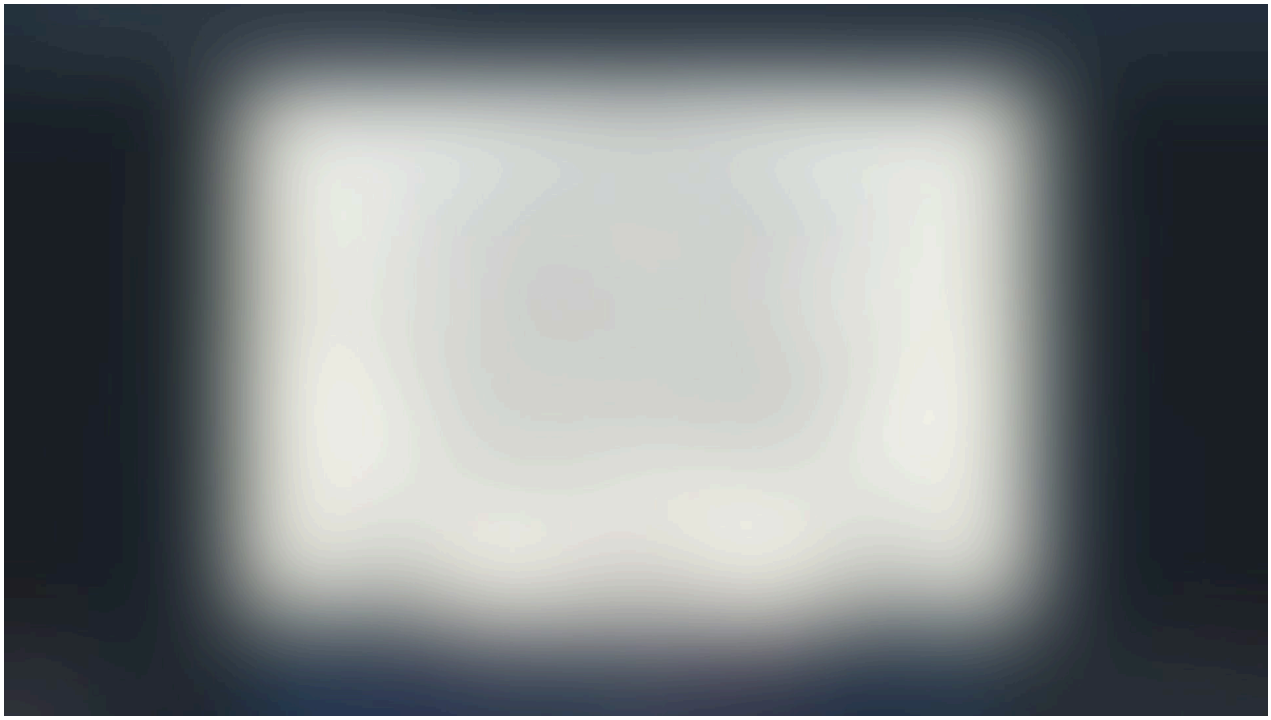
Step 7 - Repeat the request with other certificate numbers and inspect the downloaded file list.



Certificate IDOR - Download list inspected

Activity: Checked the browser download list after repeating the request with different certificate numbers. Downloaded file names and participant identifiers are masked.

Step 8 - Open a downloaded certificate to inspect whether it belongs to another participant.



Certificate IDOR - Downloaded certificate opened

Activity: Opened a downloaded certificate to inspect its contents. The full screenshot is blurred because the certificate contains identifiable participant and institution details.

Step 9 - Try a certificate number that does not exist and observe the not-found response.



Certificate IDOR - Non-existent identifier tested

Activity: Requested a certificate number that did not belong to a record and observed the server return a not-found response. Request credentials and response content are masked.

Step 10 - Repeat the request with another certificate number.



Certificate IDOR - Request repeated with another number

Activity: Repeated the certificate request with another number. Request and response panels are masked because they contain session and certificate data.

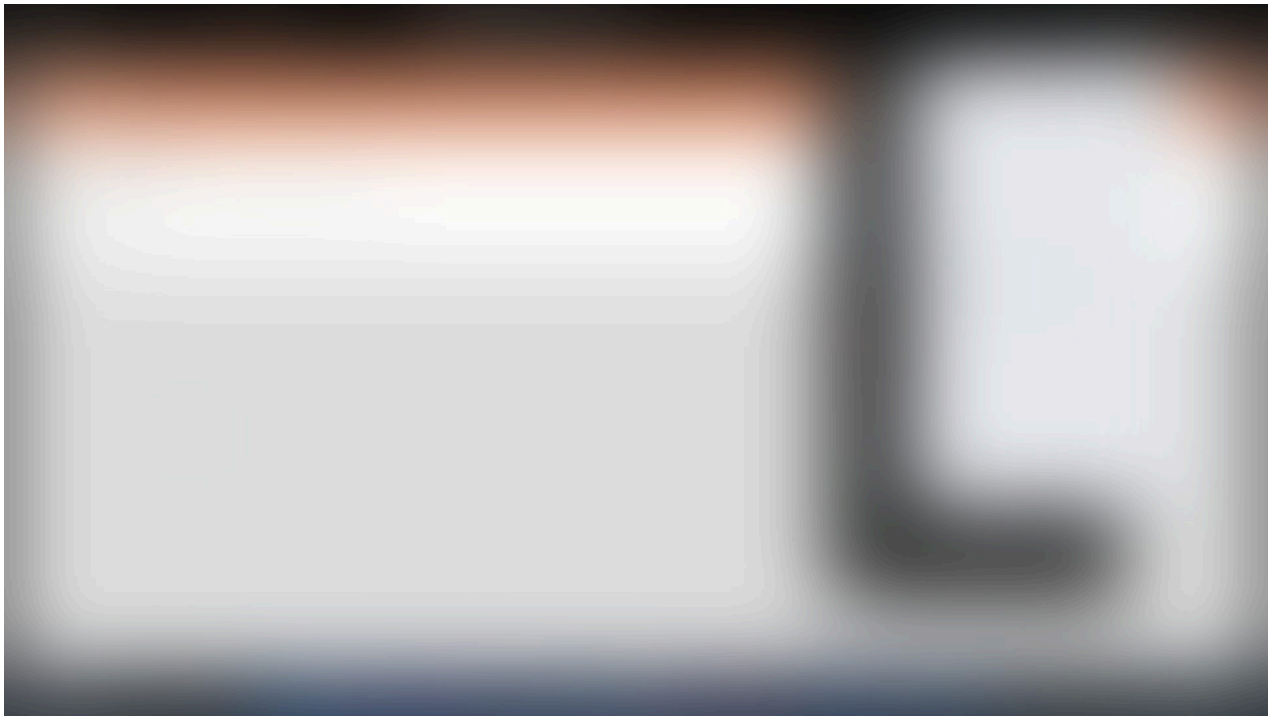
Step 11 - Observe the server response after changing only the certificate number.



Certificate IDOR - Another certificate response received

Activity: Changed only the certificate number and observed another certificate response from the server. The request and response content are masked.

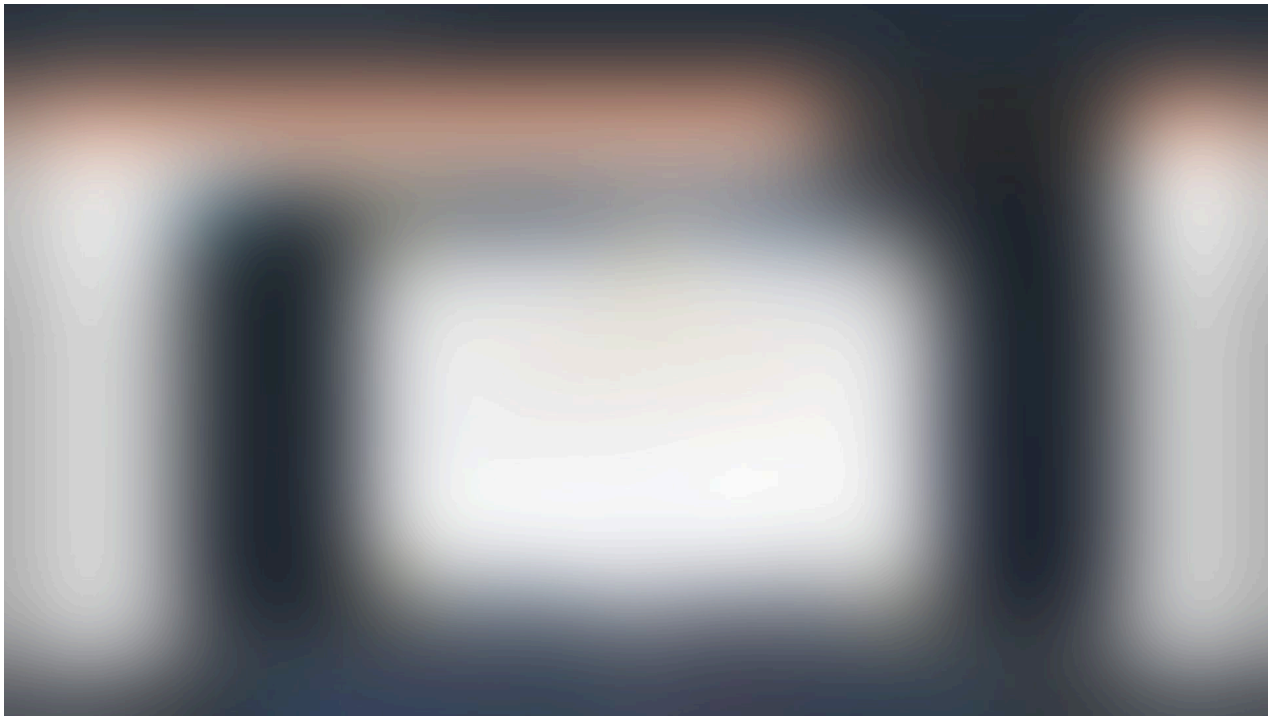
Step 12 - Open another response in the browser.



Certificate IDOR - Another response opened in browser

Activity: Opened another server response in the browser and checked the download list. Downloaded file names are masked.

Step 13 - Open the downloaded certificate and compare the result with the account used for testing.



Certificate IDOR - Another participant certificate opened

Activity: Opened another downloaded certificate after changing the certificate number. The full screenshot is blurred because it displays another participant’s identifiable certificate data.

Impact

A logged-in participant could access certificates belonging to other participants. Certificates contained personal information such as participant names, institution details and dates. Sequential certificate numbers made it easier to test other records.

Root cause

The server checked whether the requester was authenticated but did not verify that the requested certificate belonged to that requester. Certificate identifiers were sequential.

Recommended remediation

Before returning a certificate, verify on the server that it belongs to the authenticated user. Deny requests for another user’s certificate. Random identifiers may make guessing harder, but they do not replace the ownership check.

References

OWASP API Security Top 10 2023 - API1: Broken Object Level Authorization; CWE-639; WSTG-ATHZ-01